

Programming Exercises

The programming exercise focuses on applying the SOLID principles to a Python shopping system. The task is divided into two stages: reviewing the initial design and refactoring it to separate responsibilities, reduce unnecessary dependencies and improve extensibility.

The exercise provides a practical example of how SOLID principles can improve the structure of object-oriented software and make future changes easier to manage.

Designing a Real-World Application with SOLID Principles

Stage 1: Initial Code

The initial Order class manages items, calculates the Order total and processes payments. Although the program works, combining these responsibilities within the same class mixes order management with payment logic and makes the design harder to extend.

```
class Item:
    def __init__(self, name, price):
        self.name = name
        self.price = price
class Order:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)

    def calculate_total(self):
        return sum(item.price for item in self.items)

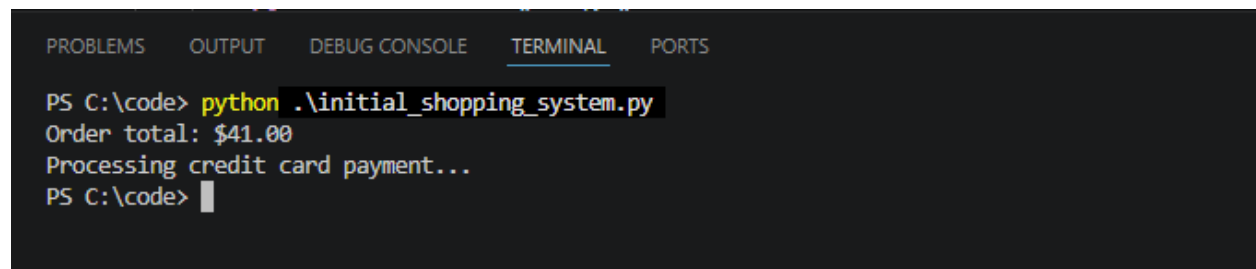
    def pay(self, payment_type):
```

```
    if payment_type == "credit":
        print("Processing credit card payment...")
    elif payment_type == "paypal":
        print("Processing PayPal payment...")
    else:
        print("Unsupported payment method.")

if __name__ == "__main__":
    order = Order()
    order.add_item(Item("Keyboard", 25.00))
    order.add_item(Item("Mouse", 16.00))

    print(f"Order total: ${order.calculate_total():.2f}")
    order.pay("credit")
```

Execution Result



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\code> python .\initial_shopping_system.py
Order total: $41.00
Processing credit card payment...
PS C:\code> |
```

The output confirms that the initial program correctly calculates the order total as **\$41.00** and processes credit card payment. Although the program functions as expected, the Order class still combines order management and payment processing responsibilities.

Design Analysis

The initial design violates the **Single Responsibility Principle (SRP)** because the Order class is responsible for both order management and payment processing. It also conflicts with the **Open/Closed Principle (OCP)** because adding a new payment method requires modifying the existing Order.pay() method. The **Dependency Inversion Principle (DIP)**

is also not followed because the order logic directly contains the payment-processing logic instead of depending on a payment abstraction.

Stage 2: Refactored SOLID Design

The refactored design separates order management, discount calculation and payment processing into focused classes and abstractions. CheckoutService coordinates the checkout process while depending on the PaymentMethod and Discount abstractions rather than specific implementations. This reduces coupling and allows new payment methods or discount strategies to be introduced without modifying the existing checkout logic.

```
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class Product:
    name: str
    price: float
class Order:

    """SRP: manages products and calculates the order total only."""

    def __init__(self):
        self.items = []

    def add_item(self, product: Product) -> None:
        self.items.append(product)

    def calculate_total(self) -> float:
        return sum(product.price for product in self.items)
class Discount(ABC):
    """Small focused abstraction for discount calculation."""

    @abstractmethod
    def apply(self, total: float) -> float:
```

```

        raise NotImplementedError

class NoDiscount(Discount):
    def apply(self, total: float) -> float:
        return total

class PercentageDiscount(Discount):
    def __init__(self, percentage: float):
        if not 0 <= percentage <= 100:
            raise ValueError("Percentage must be between 0 and 100.")
        self.percentage = percentage

    def apply(self, total: float) -> float:
        return total * (1 - self.percentage / 100)

class PaymentMethod(ABC):
    """Abstraction used by checkout service."""

    @abstractmethod
    def pay(self, amount: float) -> None:
        raise NotImplementedError

class CreditCardPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Processing credit card payment of ${amount:.2f}...")

class PayPalPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Processing PayPal payment of ${amount:.2f}...")

class CryptoPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Processing crypto payment of ${amount:.2f}...")

class CheckoutService:
    """
    DIP: depends on PaymentMethod and Discount abstractions,
    not on concrete payment or discount classes.
    """

    def __init__(self, payment_method: PaymentMethod, discount: Discount):
        self.payment_method = payment_method
        self.discount = discount

    def checkout(self, order: Order) -> float:

```

```

        subtotal = order.calculate_total()
        final_total = self.discount.apply(subtotal)

        print(f"Subtotal: ${subtotal:.2f}")
        print(f"Final total: ${final_total:.2f}")
        self.payment_method.pay(final_total)

        return final_total

if __name__ == "__main__":
    order = Order()
    order.add_item(Product("Keyboard", 25.00))
    order.add_item(Product("Mouse", 16.00))

    discount = PercentageDiscount(10) #Discount 10%

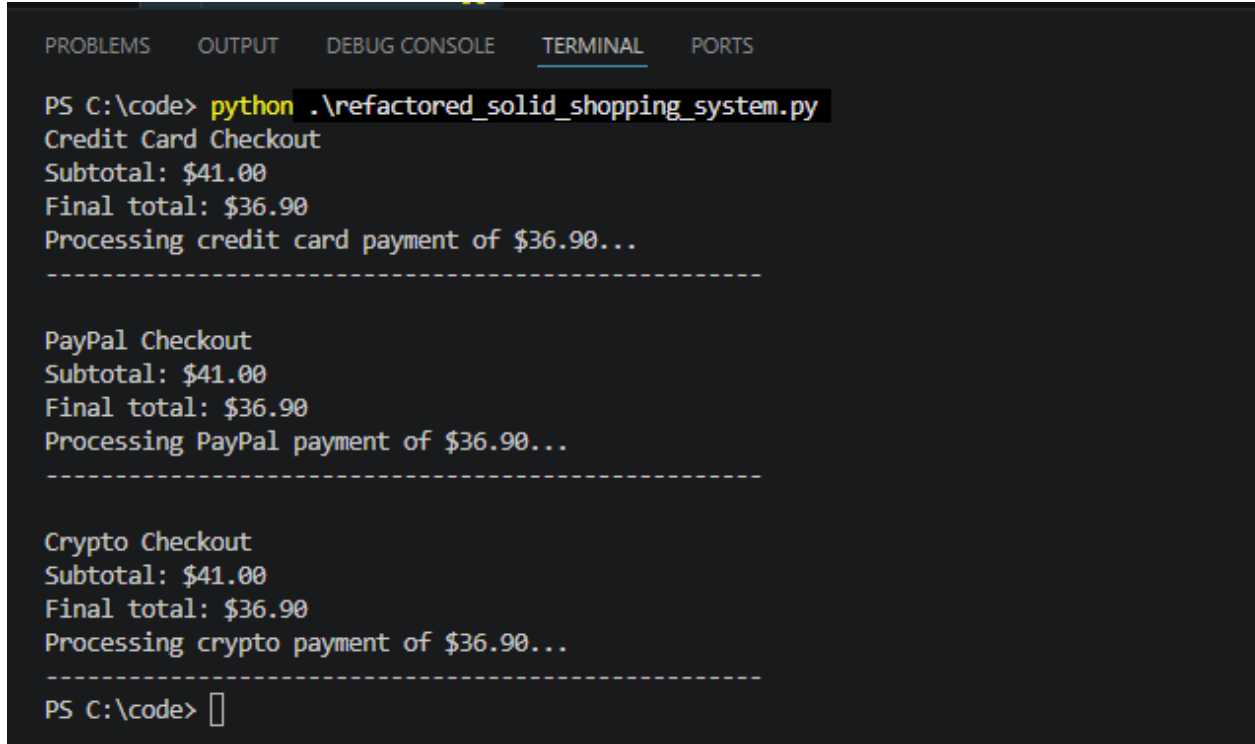
    print("Credit Card Checkout")
    credit_checkout = CheckoutService(CreditCardPayment(), discount)
    credit_checkout.checkout(order)
    print("-----")

    print("\nPayPal Checkout")
    paypal_checkout = CheckoutService(PayPalPayment(), discount)
    paypal_checkout.checkout(order)
    print("-----")

    print("\nCrypto Checkout")
    crypto_checkout = CheckoutService(CryptoPayment(), discount)
    crypto_checkout.checkout(order)
    print("-----")

```

Execution Result



```
PS C:\code> python .\refactored_solid_shopping_system.py
Credit Card Checkout
Subtotal: $41.00
Final total: $36.90
Processing credit card payment of $36.90...
-----

PayPal Checkout
Subtotal: $41.00
Final total: $36.90
Processing PayPal payment of $36.90...
-----

Crypto Checkout
Subtotal: $41.00
Final total: $36.90
Processing crypto payment of $36.90...
-----
PS C:\code> 
```

The output confirms that the order subtotal is **\$41.00** and that applying a 10% discount reduces the final total to **\$36.90**. The same order and discount are then processed successfully using credit card, PayPal and cryptocurrency payment methods without changing the checkout logic. This demonstrates that the refactored design is more flexible and maintainable while preserving the expected program behaviour.

How the SOLID Principles Are Applied

- **Single Responsibility Principle (SRP)::** Order manages products and calculates totals, payment classes process payments, and discount classes calculate discounts. Each component therefore has a focused responsibility.

- **Open/Closed Principle (OCP):** New payment methods and discount strategies can be introduced by creating new subclasses without modifying Order or CheckoutService.
- **Liskov Substitution Principle (LSP):** CreditCardPayment, PayPalPayment and CryptoPayment can be used wherever PaymentMethod is expected without changing the behaviour required by CheckoutService.
- **Interface Segregation Principle (ISP):** PaymentMethod defines only the pay() operation required by payment classes, while discount behaviour is kept in a separate focused abstraction.
- **Dependency Inversion Principle (DIP):** CheckoutService depends on the PaymentMethod and Discount abstractions rather than specific payment or discount implementations.